Student Handbook

Docker Training

Day 4 · Session 3

Docker Swarm for simple production deployments

1:00 – 2:15 · 1.25 hours · Hands-on Lab

Prerequisites: Day 2 Compose lab complete (compose-lab project) and Day 4 Session 1 hardened images available

Learning objectives

By the end of this session you will be able to:

- Initialize a Docker Swarm cluster and understand manager vs worker roles
- Deploy an existing Compose file to Swarm as a stack using the deploy: key
- Explain the difference between a service, a task, and a replica
- Inspect task placement and distribution across nodes
- Scale a service up or down on demand without downtime
- Perform a zero-downtime rolling update using start-first ordering
- Roll back a failed deployment to the last known-good version
- Understand overlay networks and the Swarm routing mesh

Key concepts

`Swarm mode` Built into the Docker engine — no extra install. docker swarm init turns the current engine into a manager node.	**`Manager node`** Holds cluster state, schedules tasks onto workers, exposes the Swarm API. Use an odd number (1, 3, 5) for quorum.
`Worker node` Executes containers (tasks) assigned by managers. Cannot make scheduling decisions itself.	**`Stack`** A Compose file deployed to Swarm with docker stack deploy. Same YAML you know — Swarm adds the deploy: key.
`Service` A Swarm-managed definition of a task: which image, how many replicas, update policy. One service runs many replicas.	**`Replica`** One running container instance of a service. docker service scale api=3 means 3 replicas running simultaneously.
`Overlay network` A virtual network spanning multiple physical hosts. Containers on different machines reach each other by service name.	**`Rolling update`** Swarm replaces replicas a few at a time per update_config, keeping old replicas serving until new ones are healthy.

Why Swarm fits here

Swarm reuses the exact Compose file syntax from Day 2 with one addition — the deploy: key. It is the simplest path from "it works in Compose on my laptop" to "it runs with replicas and rolling updates," before the added complexity of full Kubernetes in the next session.

Hands-on lab steps

1 Initialize Swarm mode

Turn your existing Docker engine into a single-node Swarm manager

Initialize the swarm

```
docker swarm init
```

Expected output:

```
Swarm initialized: current node (a1b2c3d4e5f6) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-xxxxx 192.168.1.10:2377

To add a manager to this swarm, run 'docker swarm join-token manager'
```

If you have multiple machines — get the join token

```
docker swarm join-token worker
```

Expected output:

```
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-xxxxx-yyyyy 192.168.1.10:2377
```

On a second machine — join as a worker (skip if single-node lab)

```
# Run this ON THE WORKER MACHINE, using the token from above
docker swarm join --token SWMTKN-1-xxxxx-yyyyy 192.168.1.10:2377
```

Expected output:

```
This node joined a swarm as a worker.
```

Verify the cluster

```
docker node ls
```

Expected output:

```
ID            HOSTNAME    STATUS    AVAILABILITY   MANAGER STATUS
a1b2c3d4 *    manager1    Ready     Active          Leader
b2c3d4e5      worker1     Ready     Active
```

Swarm is built in, not bolted on

There is no separate binary to install. docker swarm init promotes the current engine to a manager and generates a cluster identity (mTLS certificates handled automatically). If only one laptop is available, that single node acts as both manager and worker — Swarm schedules containers onto it normally.

Trainer tip

If trainees are on individual laptops without network access to each other, have everyone run a single-node swarm. Pair up only those on the same Wi-Fi/LAN to demo true multi-node joining if time allows.

2 Deploy your Compose file as a Swarm stack

The same YAML from Day 2 — Swarm adds a deploy: section

Reuse the compose-lab project from Day 2

```
cd ~/docker-training/compose-lab

# Add a 'deploy:' block to each service in docker-compose.yml
# (Swarm reads deploy: — plain docker compose up ignores it harmlessly)
```

docker-compose.yml — add deploy: under each service

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - appnet
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure
      placement:
        constraints:
          - node.role == manager   # Keep the DB pinned to a stable node

  api:
    image: weatherapi:1.0
    environment:
      ConnectionStrings__DefaultConnection:
"Host=db;Port=5432;Database=${DB_NAME};Username=${DB_USER};Password=${DB_PASS}"
    networks:
      - appnet
    ports:
      - "8080:8080"
    deploy:
      replicas: 2
      update_config:
        parallelism: 1
        delay: 5s
        order: start-first
      restart_policy:
        condition: on-failure

volumes:
  pgdata:

networks:
  appnet:
    driver: overlay   # Swarm requires 'overlay' instead of the default bridge
```

Deploy the stack

```
docker stack deploy -c docker-compose.yml myapp
```

Expected output:

```
Creating network myapp_appnet
Creating service myapp_db
Creating service myapp_api
```

Verify the stack and its services

```
docker stack ls
docker stack services myapp
```

Expected output:

```
NAME    SERVICES
myapp   2

ID          NAME         MODE         REPLICAS   IMAGE
a1b2c3      myapp_db     replicated   1/1        postgres:16
b2c3d4      myapp_api    replicated   2/2        weatherapi:1.0
```

Stacks reuse Compose syntax almost entirely

The only Swarm-specific addition is the deploy: key, which a plain docker compose up ignores. networks: must use driver: overlay instead of the default bridge — overlay networks work across multiple physical hosts, while bridge networks are single-host only.

Why placement constraints for the database

node.role == manager pins the postgres replica to a specific node so its volume data stays in one predictable location. In a real multi-node cluster you would normally use a proper distributed storage driver instead of relying on node placement, but this constraint is a simple way to demonstrate the concept in a training lab.

3 Inspect running tasks and service distribution

See exactly which node each replica landed on

List all tasks (replicas) for a specific service

```
docker service ps myapp_api
```

Expected output:

```
ID          NAME           IMAGE              NODE       DESIRED STATE   CURRENT STATE
x1y2z3      myapp_api.1    weatherapi:1.0     manager1   Running         Running 2
minutes ago
a4b5c6      myapp_api.2    weatherapi:1.0     manager1   Running         Running 2
minutes ago
```

Get full service configuration in readable form

```
docker service inspect myapp_api --pretty
```

Expected output:

```
ID:             b2c3d4e5f6a7
Name:           myapp_api
Service Mode:   Replicated
 Replicas:      2
```

```
 UpdateConfig:
  Parallelism:   1
  Delay:         5s
  Order:         start-first
 RestartPolicy:
  Condition:     on-failure
 Image:          weatherapi:1.0
```

Test the API — Swarm load-balances across replicas automatically

```
for i in 1 2 3 4; do curl -s http://localhost:8080/weatherforecast | head -c 50;
echo; done
```

Expected output:

```
[{"date":"2026-06-20","temperatureC":18,"summary"...
[{"date":"2026-06-20","temperatureC":22,"summary"...
[{"date":"2026-06-20","temperatureC":18,"summary"...
[{"date":"2026-06-20","temperatureC":22,"summary"...
```

Simulate a replica failing — Swarm self-heals

```
docker ps --filter "name=myapp_api"
docker kill <container-id-of-one-replica>

# Watch Swarm immediately reschedule a replacement
docker service ps myapp_api
```

Expected output:

```
ID          NAME               IMAGE             NODE       CURRENT STATE
x1y2z3      myapp_api.1        weatherapi:1.0    manager1   Running 3 minutes ago
a4b5c6       \_ myapp_api.2    weatherapi:1.0    manager1   Shutdown 2 seconds ago
m7n8o9      myapp_api.2        weatherapi:1.0    manager1   Running 1 second ago
# A NEW task (m7n8o9) was created automatically to replace the killed one
```

Self-healing is automatic and continuous

Swarm constantly compares desired state (replicas: 2) against actual state. If a replica dies for any reason — crash, node failure, manual kill — a manager immediately schedules a replacement. This happens with zero commands from you; it is the core value proposition of any orchestrator over plain docker run.

Routing mesh

You only published port 8080 once, but requests get distributed across both replicas. Swarm's routing mesh listens on the published port on EVERY node and load-balances to whichever node currently has a healthy replica, using IPVS load balancing internally.

4 Scale a service up and down

Change replica count instantly without touching the compose file

Scale the API service to 3 replicas

```
docker service scale myapp_api=3
```

Expected output:

```
myapp_api scaled to 3
```

```
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service converged
```

Confirm the new replica count

```
docker service ls
docker service ps myapp_api
```

Expected output:

```
ID        NAME        MODE        REPLICAS   IMAGE
b2c3d4    myapp_api   replicated  3/3        weatherapi:1.0

ID       NAME          NODE       CURRENT STATE
x1y2z3   myapp_api.1   manager1   Running 5 minutes ago
a4b5c6   myapp_api.2   manager1   Running 5 minutes ago
p9q8r7   myapp_api.3   manager1   Running 10 seconds ago
```

Scale multiple services at once

```
docker service scale myapp_api=4 myapp_db=1
```

Scale back down for the next exercise

```
docker service scale myapp_api=2
```

Expected output:

```
myapp_api scaled to 2
overall progress: 2 out of 2 tasks
verify: Service converged
```

Scaling is live and in-place

No rebuild, no redeploy, no downtime for existing replicas. Swarm either starts new tasks (scaling up) or gracefully stops the highest-numbered tasks (scaling down) to reach the desired count.

Trainer tip

Run the load test command from Step 3 again right after scaling to 3, and watch a third unique response appear in rotation. This makes "scaling" feel concrete rather than abstract.

5 Perform a zero-downtime rolling update

Deploy a new image version while the service keeps serving traffic

Make a small visible change and build a new version of the API

```
cd ~/docker-training/WeatherApi

# Make a tiny change so the rolling update is visible — e.g. edit the
# summary text in the weather forecast generator, then rebuild:
docker build -t weatherapi:2.0 .
```

In one terminal — start a continuous request loop

```
# Run this in a SEPARATE terminal and leave it running through the next step
while true; do
  curl -s -o /dev/null -w "%{http_code} " http://localhost:8080/weatherforecast
  sleep 0.5
done
```

Expected output:

```
200 200 200 200 200 200 200 200 200 200 200 ...
# Watch this stream during the rolling update below — it should
# never show a non-200 code or a connection error
```

In another terminal — trigger the rolling update

```
docker service update \
  --image weatherapi:2.0 \
  --update-parallelism 1 \
  --update-delay 5s \
  myapp_api
```

Expected output:

```
myapp_api
overall progress: 2 out of 2 tasks
1/2: running   [==================================================>]
2/2: running   [==================================================>]
verify: Service converged
```

Watch the rollout progress in real time

```
watch docker service ps myapp_api
```

Expected output:

```
ID       NAME              IMAGE             CURRENT STATE
m1n2o3   myapp_api.1       weatherapi:2.0    Running 8 seconds ago
x1y2z3    \_ myapp_api.1   weatherapi:1.0    Shutdown 10 seconds ago
a4b5c6   myapp_api.2       weatherapi:1.0    Running 5 minutes ago
# Task 1 updated first; task 2 will update next after the delay
```

Confirm both replicas are now on the new version

```
docker service inspect myapp_api --format
'{{.Spec.TaskTemplate.ContainerSpec.Image}}'
```

Expected output:

```
weatherapi:2.0@sha256:...
```

Why this is genuinely zero-downtime

order: start-first (set in Step 2's compose file) tells Swarm to start the NEW replica and wait for it to be running before stopping the OLD one. update-parallelism 1 updates one replica at a time; update-delay 5s waits between each one, giving you time to catch problems before the whole fleet is updated.

The moment that matters

The continuous curl loop from this step should show unbroken 200 responses throughout the entire rollout. This is the single most convincing demo of the whole session — let it run long enough for trainees to genuinely watch nothing break.

6 Roll back a bad deployment

Instantly revert to the previous working version

Simulate a bad deploy — push a version that fails health checks

```
# Imagine weatherapi:3.0 has a bug that crashes on startup
docker service update --image weatherapi:3.0-broken myapp_api
```

Expected output:

```
myapp_api
overall progress: 0 out of 2 tasks
1/2: starting    [============>                                          ]
2/2: starting    [============>                                          ]
# Tasks fail to reach 'running' — something is wrong
```

Check what is happening

```
docker service ps myapp_api
```

Expected output:

```
ID       NAME              IMAGE                   CURRENT STATE
p1q2r3   myapp_api.1       weatherapi:3.0-broken Failed 5 seconds ago
 \_ x1y2z3  myapp_api.1     weatherapi:2.0         Shutdown 20 seconds ago
# The new image is crash-looping
```

Roll back to the previous working version immediately

```
docker service rollback myapp_api
```

Expected output:

```
myapp_api
rollback: manually requested rollback
overall progress: 2 out of 2 tasks
verify: Service converged
```

Confirm the rollback restored the working version

```
docker service inspect myapp_api --format
'{{.Spec.TaskTemplate.ContainerSpec.Image}}'
```

Expected output:

```
weatherapi:2.0@sha256:...
# Back to the last known-good image
```

Clean up at the end of the session

```
# Remove the entire stack (services, networks — volumes are kept)
docker stack rm myapp


# Leave the swarm if you want to return to standalone Docker
docker swarm leave --force
```

Expected output:

```
Removing service myapp_api
Removing service myapp_db
Removing network myapp_appnet
```

Important nuance

docker service rollback only goes back ONE step — it cannot step back multiple versions in sequence. For more sophisticated rollback history, track image tags/digests in your own deployment records (e.g. git tags, CI/CD release logs) alongside Swarm.

Session outcome

You can initialize a Swarm cluster, deploy a Compose file as a stack, scale services on demand, perform genuinely zero-downtime rolling updates, and instantly roll back a bad deployment. This is a complete, simple production deployment workflow — no Kubernetes complexity required.

Quick reference — cluster commands

Command	What it does
`docker swarm init`	Initialize a single-node swarm; current engine becomes manager
`docker swarm join-token worker`	Get the token needed to join a worker node to the swarm
`docker swarm join --token ... IP:2377`	Join a machine to the swarm using a token (run on that machine)
`docker node ls`	List all nodes in the cluster with status and role
`docker swarm leave`	Leave the swarm (run on the node leaving)
`docker swarm leave --force`	Force the last manager to leave (destroys the swarm)

Quick reference — stack and service commands

Command	What it does
`docker stack deploy -c file.yml name`	Deploy a compose file as a stack
`docker stack ls`	List all stacks
`docker stack services name`	List services in a stack
`docker stack ps name`	List all running tasks (replicas) for a stack
`docker stack rm name`	Remove an entire stack (volumes are kept)
`docker service ls`	List all services across the cluster
`docker service scale name=N`	Scale a specific service to N replicas
`docker service ps name`	Inspect replica placement across nodes
`docker service update --image img name`	Update a service image (triggers rolling update)
`docker service rollback name`	Roll back to the previous version
`docker service inspect name --pretty`	View detailed, human-readable service configuration

Compose-to-Swarm key reference

Key	Purpose and notes

`deploy.replicas:`	Number of container instances (tasks) to run for this service.
`deploy.restart_policy.condition:`	on-failure, any, or none. Controls when Swarm restarts a failed task.
`deploy.update_config.parallelism:`	How many replicas to update at once during a rolling update.
`deploy.update_config.delay:`	Wait time between updating each batch of replicas.
`deploy.update_config.order:`	start-first (zero downtime) or stop-first (default, briefly drops capacity).
`deploy.placement.constraints:`	Restrict where tasks can run, e.g. node.role == manager.
`networks.<name>.driver: overlay`	Required for multi-host networking in Swarm (replaces default bridge).

Common mistakes to avoid

Mistake 1 — Using the default bridge network in a stack

A plain bridge network only works on a single host. Any service-to-service communication across multiple nodes requires driver: overlay declared at the top level of the compose file.

Mistake 2 — Forgetting order: start-first for zero-downtime updates

Without order: start-first, Swarm defaults to stop-first — it stops the old replica before starting the new one, causing a brief capacity drop. For genuinely zero-downtime updates, always set order: start-first in update_config.

Mistake 3 — Expecting docker service rollback to undo multiple deploys

Rollback only reverts ONE update — the immediately previous TaskTemplate. If you have deployed several versions since the last good one, a single rollback will not reach further back. Track release versions externally for deeper history.

Mistake 4 — Running docker stack rm and expecting volumes to be deleted too

docker stack rm removes services and networks but keeps named volumes, just like docker compose down without -v. If you need a completely clean slate, remove volumes separately with docker volume rm.

Coming up — Session 4: Capstone project

In the final session, teams bring together everything from all four days into one end-to-end containerized solution: hardened Dockerfiles, full Compose stack with monitoring, CI/CD pipeline, and a Swarm deployment with a live rolling update demo.

- Fork the starter repo and apply production-ready, hardened Dockerfiles
- Write a complete docker-compose.yml with db, api, ui, reverse proxy, and monitoring
- Wire a GitHub Actions pipeline that builds and pushes images on every commit
- Deploy the finished stack to Swarm and demonstrate a zero-downtime rolling update

Keep your swarm and stack ready

If your machine is still part of a swarm with the myapp stack deployed, you can reuse this exact setup as the deployment target for the capstone project.